Deploying a Streamlit App on GitHub Codespaces

This guide walks you through the entire process of deploying a Streamlit application using GitHub Codespaces. Codespaces gives you a complete cloud-based development environment right in your browser, so you do not need to install anything locally. All you need is a GitHub account and a repository with your Streamlit app.

The example used here is a Customer Churn Prediction App built with Streamlit and powered by an XGBoost model. The same steps apply to any Streamlit application.

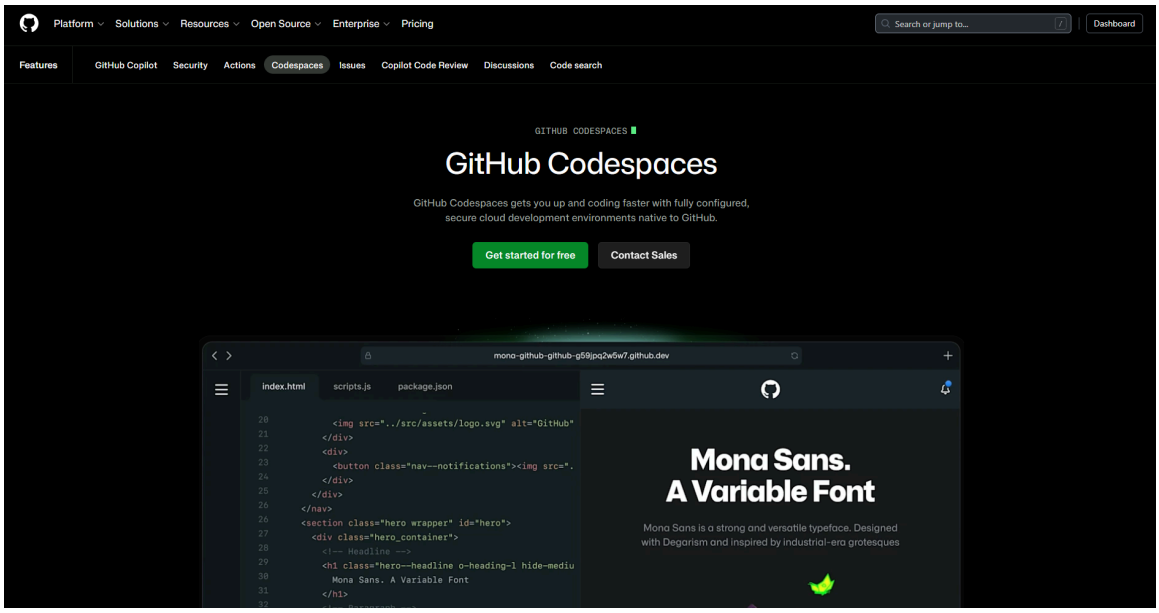
Prerequisites:

1.) Ensure that you are logged in to your GitHub account.
2.) Ensure that the Streamlit app files are available in your GitHub repository.

Step 1: Open the GitHub Codespaces Page

Go to github.com/features/codespaces in your browser. This is the main landing page for GitHub Codespaces. You will see an overview of what Codespaces offers, which is basically a fully configured cloud development environment that runs right inside your browser.

Click on **"Get started for free"**



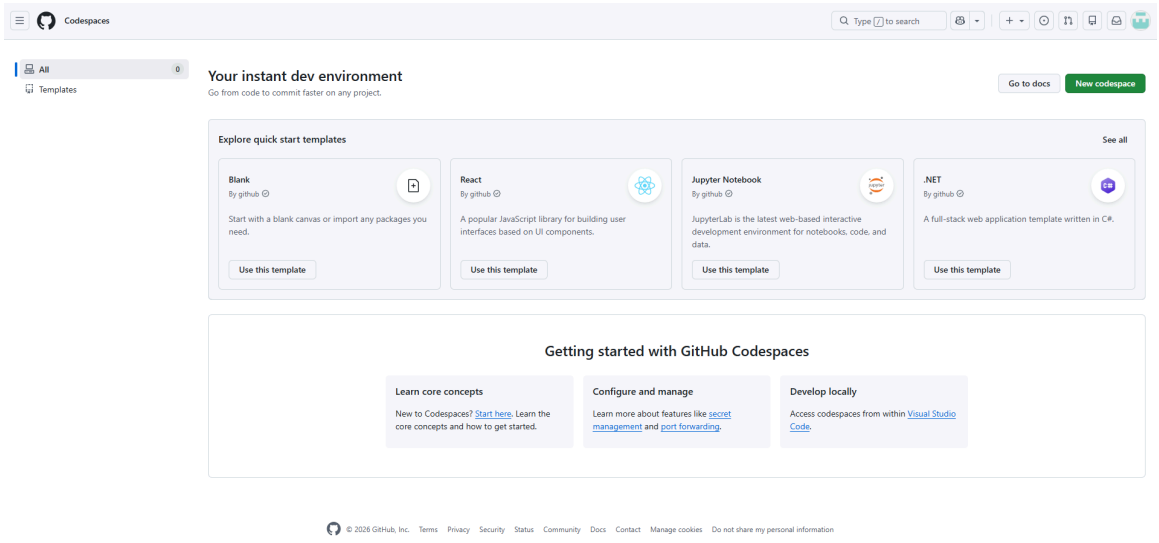
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 2: View Your Codespaces Dashboard

Once you are on the Codespaces dashboard, you will see a page titled **"Your instant dev environment."** If this is your first time, there will be no active codespaces listed.

You will also notice some quick start templates like Blank, React, Jupyter Notebook, and .NET. These are handy for starting from scratch, but since we are deploying an existing Streamlit app from a repository, we will create a new codespace from a repo instead.

Click the green "New codespace" button in the top right corner.

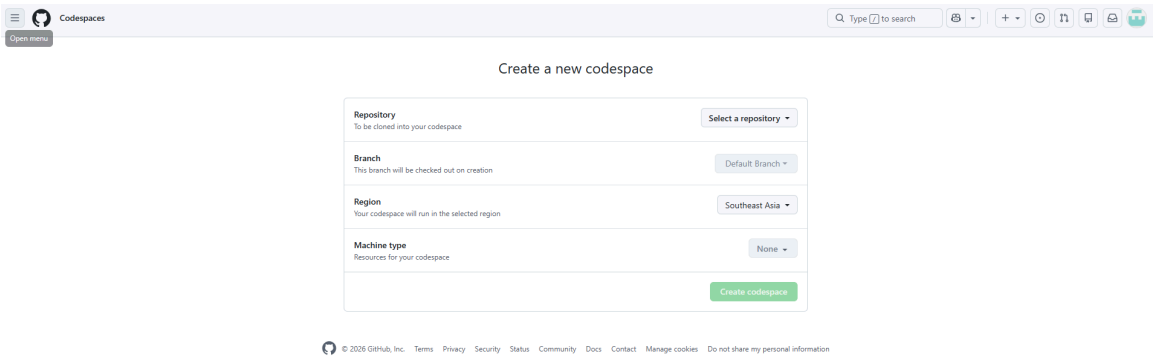


Step 3: Start Creating a New Codespace

You will be taken to the **"Create a new codespace"** page. Here you need to fill in a few details. The form has fields for Repository, Branch, Region, and Machine type.

Start by clicking the **"Select a repository"** dropdown and search for or select the repository that contains your Streamlit app.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 4: Configure and Create the Codespace

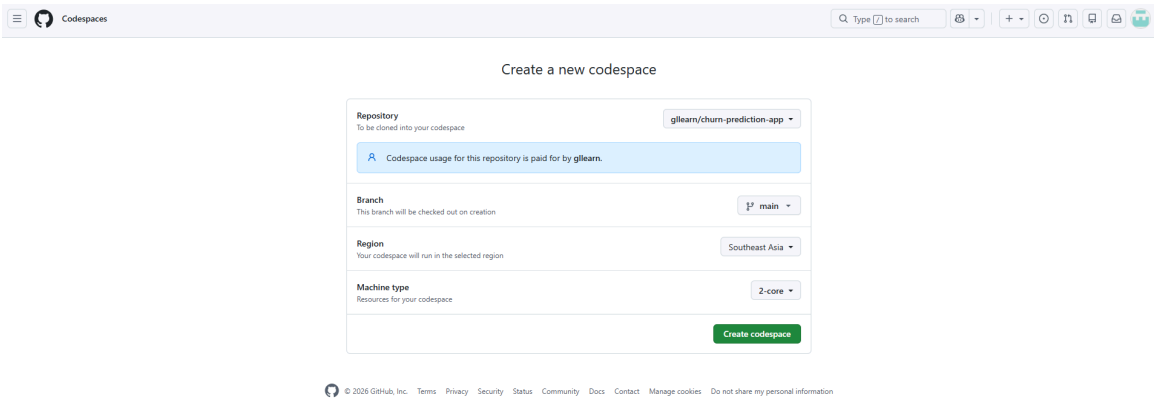
After selecting your repository (in this example, gllearn/churn-prediction-app), the form will automatically fill in some defaults. The branch will be set to "main," the region will be picked based on your location (here it shows Southeast Asia), and the machine type will default to 2-core.

You can leave the defaults as they are for most use cases. The 2-core machine is enough for running a Streamlit app. If your app is heavier, you can always bump this up later.

Note: With a 2-core machine, you will get 60 hours of usage per month, whereas with a 4-core machine, you will get 30 hours of usage per month. We highly recommend sticking with the 2-core machine, as it is more than sufficient for this case study.

Click the green **"Create codespace"** button to spin up your environment.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



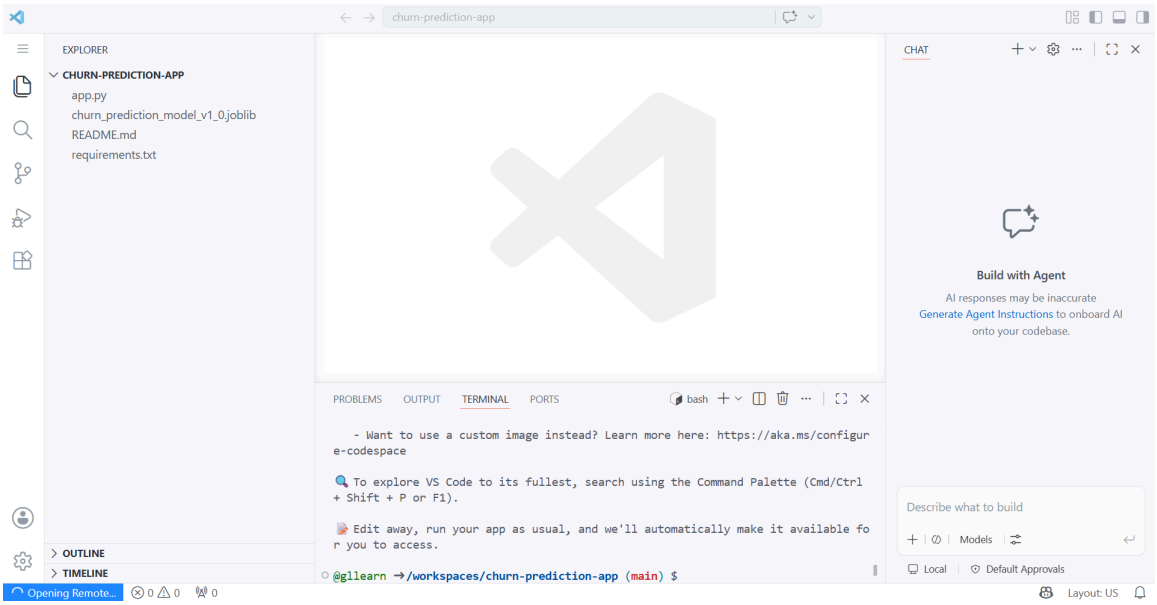
Step 5: Wait for the Codespace to Load

GitHub will now build and open your codespace. This takes a minute or two depending on the repository size. You will see VS Code loading directly in your browser.

Once it is ready, the file explorer on the left will show all the files from your repository.

In this case, you can see app.py (the main Streamlit script), churn_prediction_model_v1_0.joblib (the saved ML model), README.md, and requirements.txt.

There will be four tabs opened at the bottom: **PROBLEMS**, **OUTPUT**, **TERMINAL**, and **PORTS**. Select **TERMINAL** if it is not selected by default.



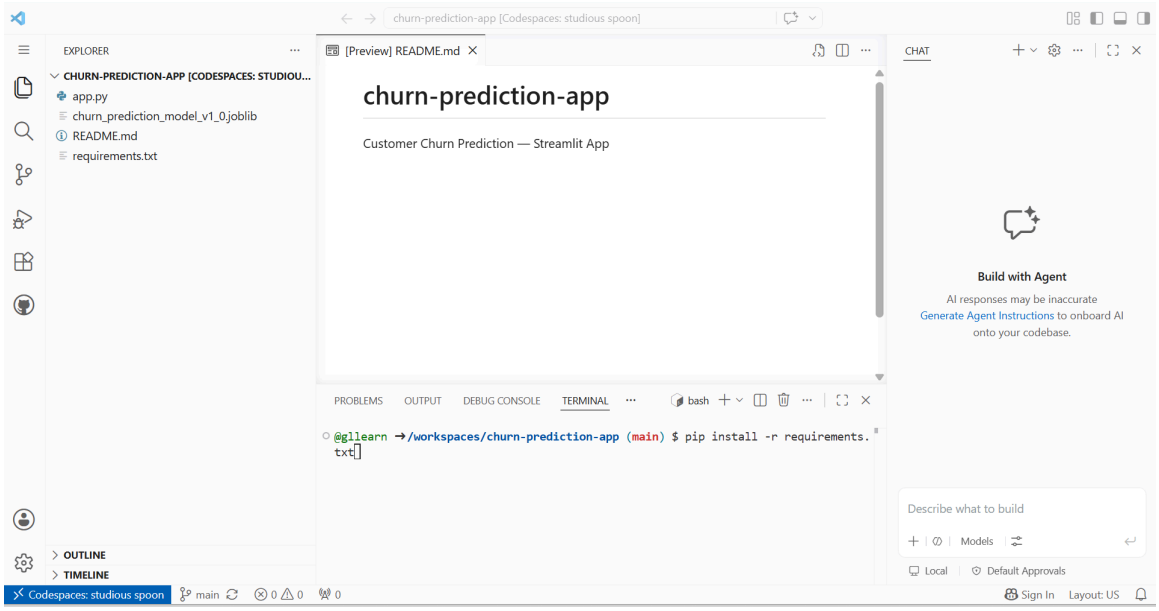
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 6: Install the Required Dependencies

Type the following command and press Enter:

```
pip install -r requirements.txt
```

This tells pip to install all the Python packages listed in the requirements.txt file.



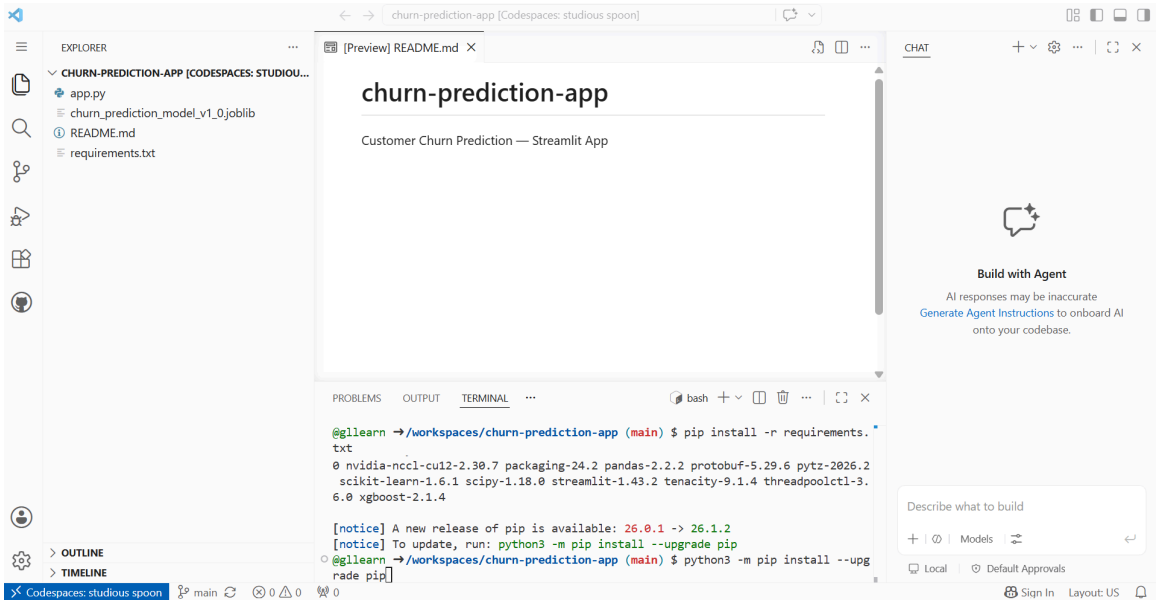
Step 7: Confirm the Packages Are Installed

You will see the terminal output showing all the packages being downloaded and installed. In this example, the packages include pandas, scikit-learn, streamlit, xgboost, scipy, and a few others.

If you see a notice that a newer version of pip is available, you can go ahead and upgrade it. This is optional but good practice. Run this command:

```
python3 -m pip install --upgrade pip
```

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

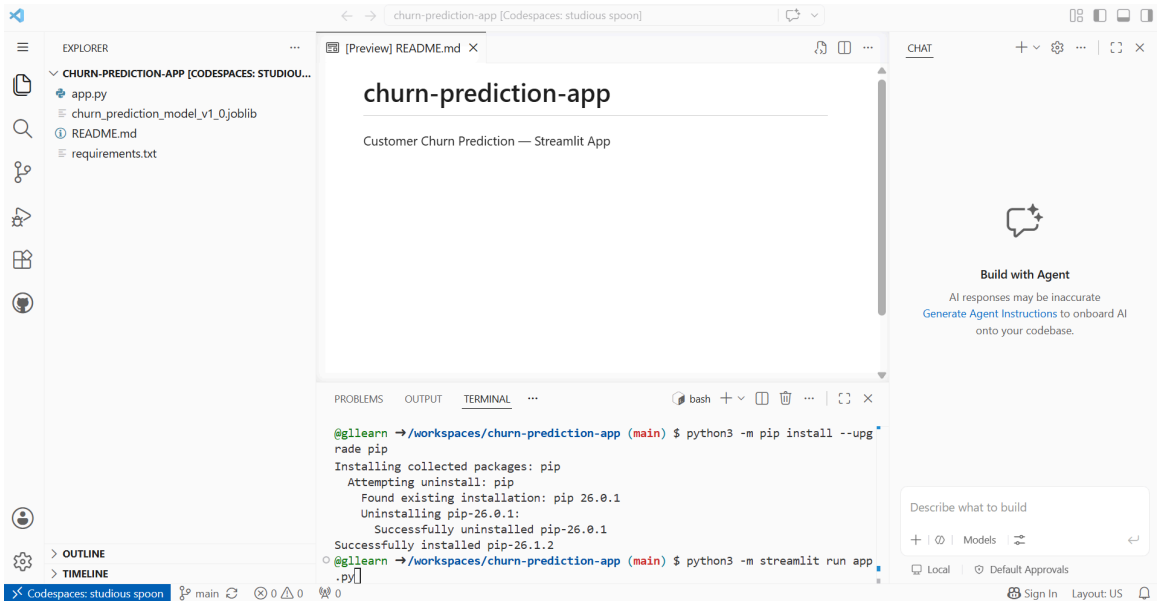


Step 8: Run the Streamlit App

Now it is time to launch the app. Type the following command in the terminal:

```
python3 -m streamlit run app.py
```

This tells Python to run Streamlit as a module and point it at your app.py file.



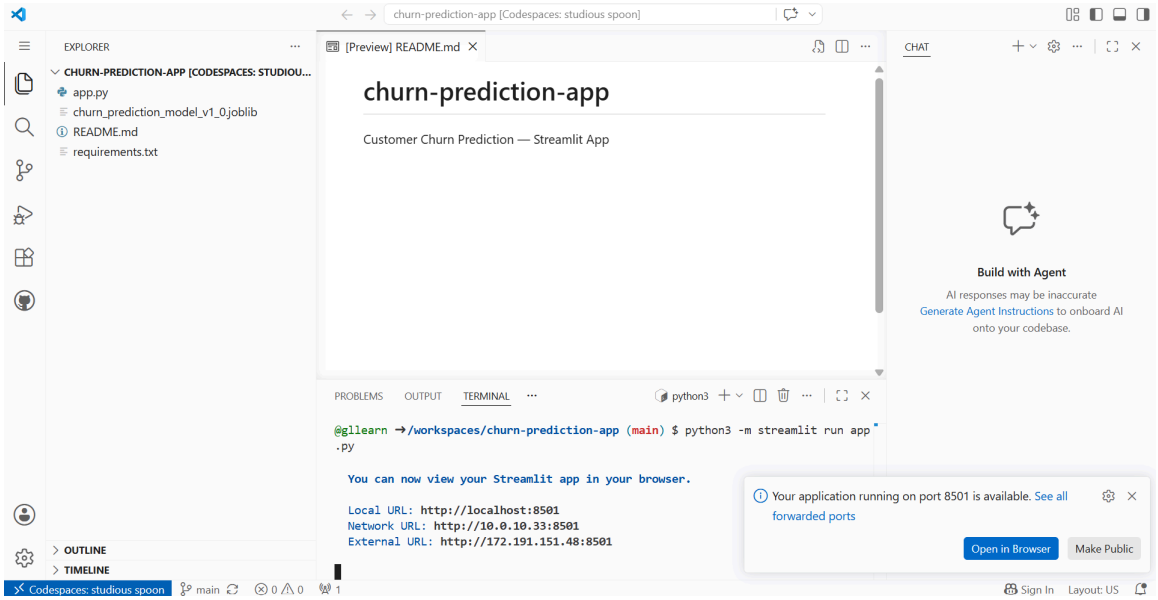
Step 9: Streamlit Starts Running

If everything is set up correctly, the terminal will show a message saying "You can now view your Streamlit app in your browser" along with a few URLs (Local URL, Network URL, and External URL).

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

At the same time, a notification will pop up in the bottom right corner of VS Code saying "Your application running on port 8501 is available." You will see two buttons: **"Open in Browser"** and **"Make Public."**

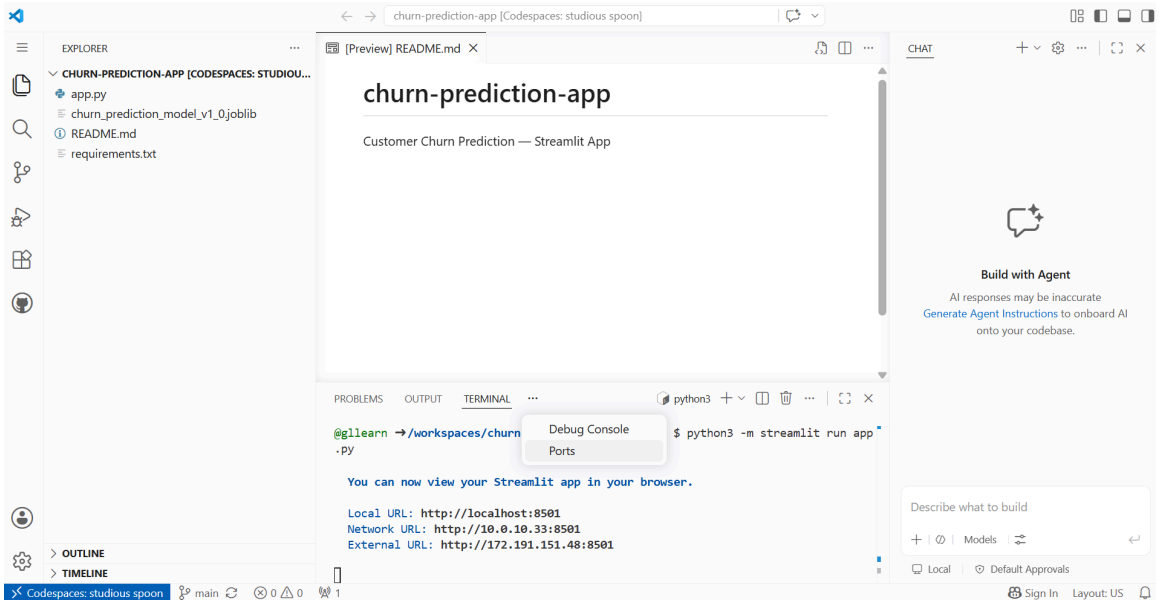
Click **"Make Public"** first so that the app URL can be accessed from outside the codespace. Then click **"Open in Browser"** to view your app.



Step 10: Check the Ports Panel

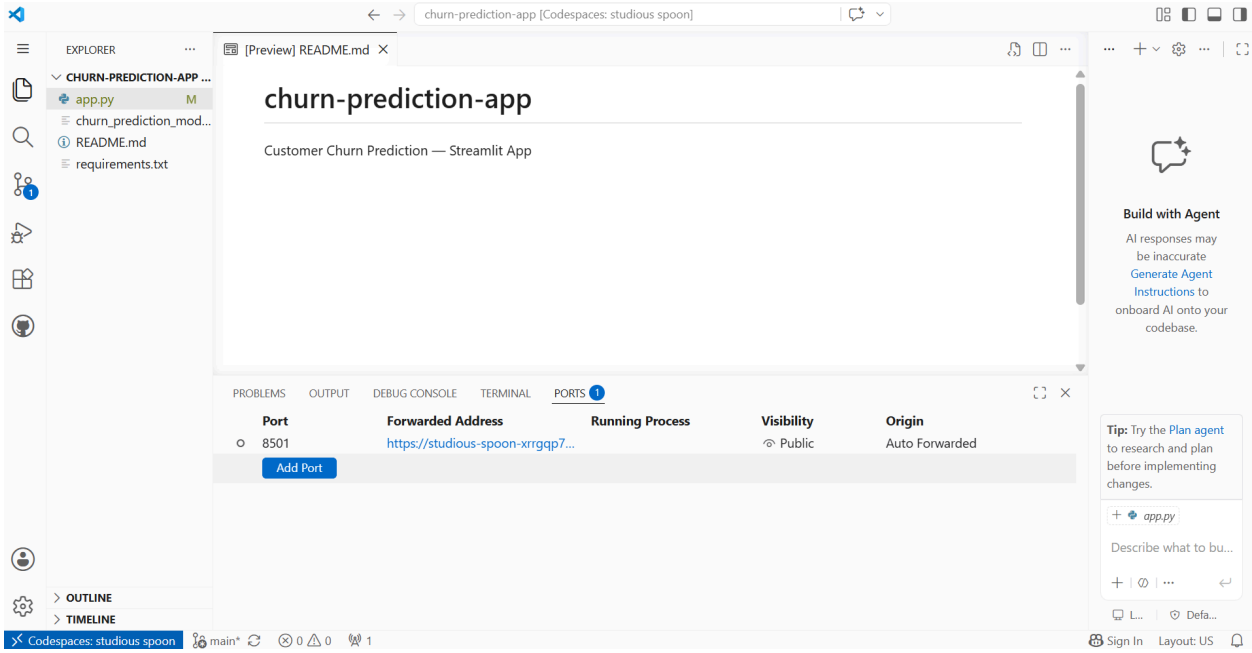
You can also check the port status from the terminal panel. Click on the three dots ("...") next to the terminal tab, and you will see options for Debug Console and Ports. Click **"Ports"** to open the Ports panel.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 11: Verify Port Visibility Is Set to Public

In the Ports panel, you will see port 8501 listed. The important columns to check are Visibility and Running Process. Make sure the visibility is set to "Public" so that anyone with the link can access your app.



Then, hover over the link mentioned under the **Forwarded Address** and click on **“Follow Link”** to open the Streamlit web app.

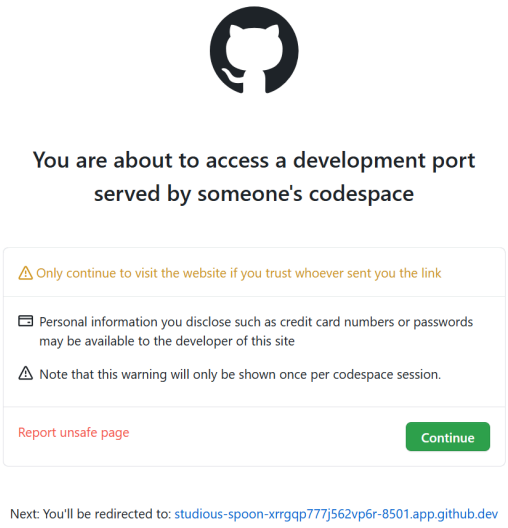
This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Step 12: GitHub Development Port Warning

When you (or anyone else) open the forwarded URL for the first time, GitHub will show a warning page. It says **"You are about to access a development port served by someone's codespace."**

This is just a safety notice letting you know that the content is served from a development environment. It also warns that personal information you enter may be visible to the developer.

Click the green **"Continue"** button to proceed to the app.

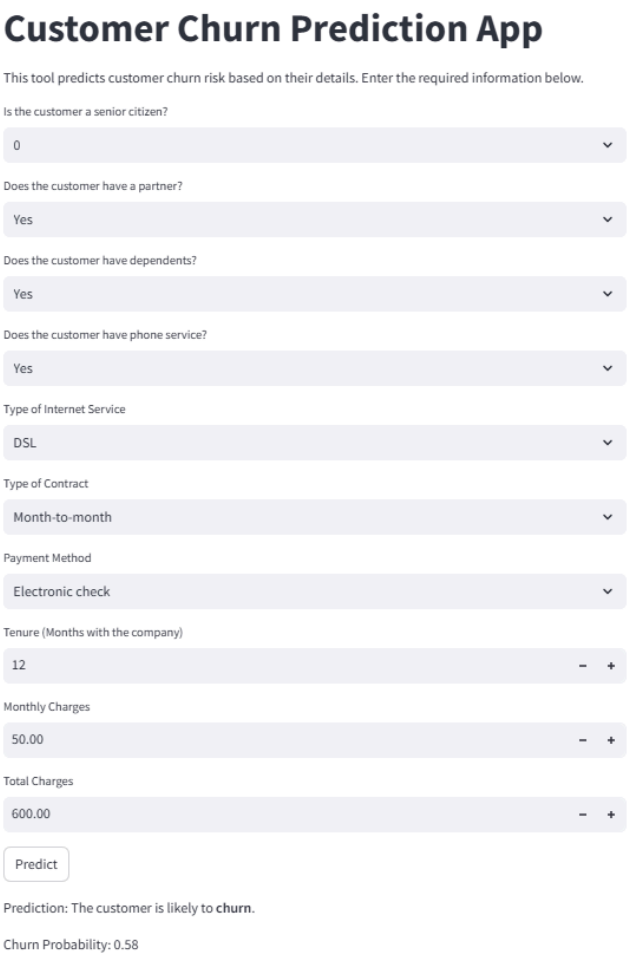


Step 13: Your Streamlit App Is Live

And that is it. Your Streamlit app is now running and accessible through the browser. In this example, the Customer Churn Prediction App is fully functional. You can see the input fields (senior citizen status, partner, dependents, phone service, internet service type, contract type, payment method, tenure, monthly charges, and total charges) and a Predict button at the bottom.

After clicking Predict with the sample inputs shown, the app displays the result: "The customer is likely to churn" with a churn probability of 0.58.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

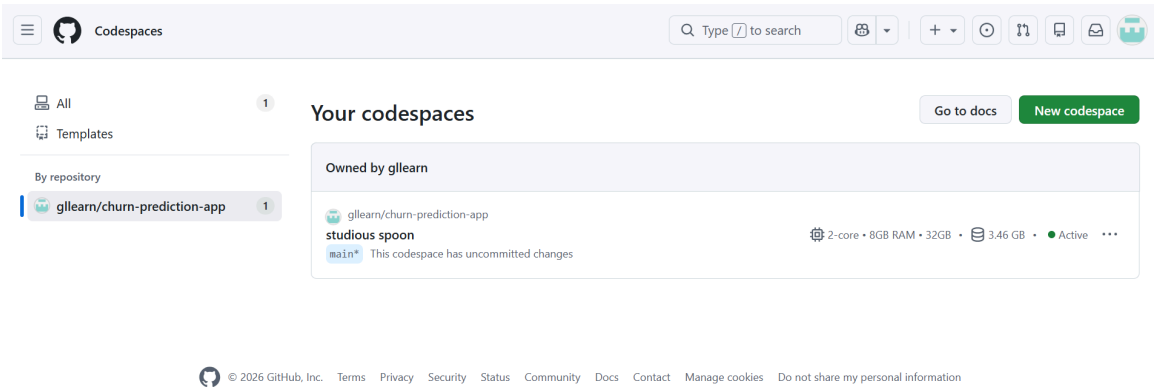


Step 14: View Your Active Codespace in the Dashboard

If you go back to github.com/features/codespaces, you will see your newly created codespace listed under **"Your codespaces."** It shows the repository name (gllearn/churn-prediction-app), the codespace name (in this case "studious spoon"), the branch (main), machine specs (2-core, 8GB RAM, 32GB), disk usage, and the status (Active).

From here you can click on the codespace name to reopen it in the browser at any time.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.



Step 15: Manage Your Codespace

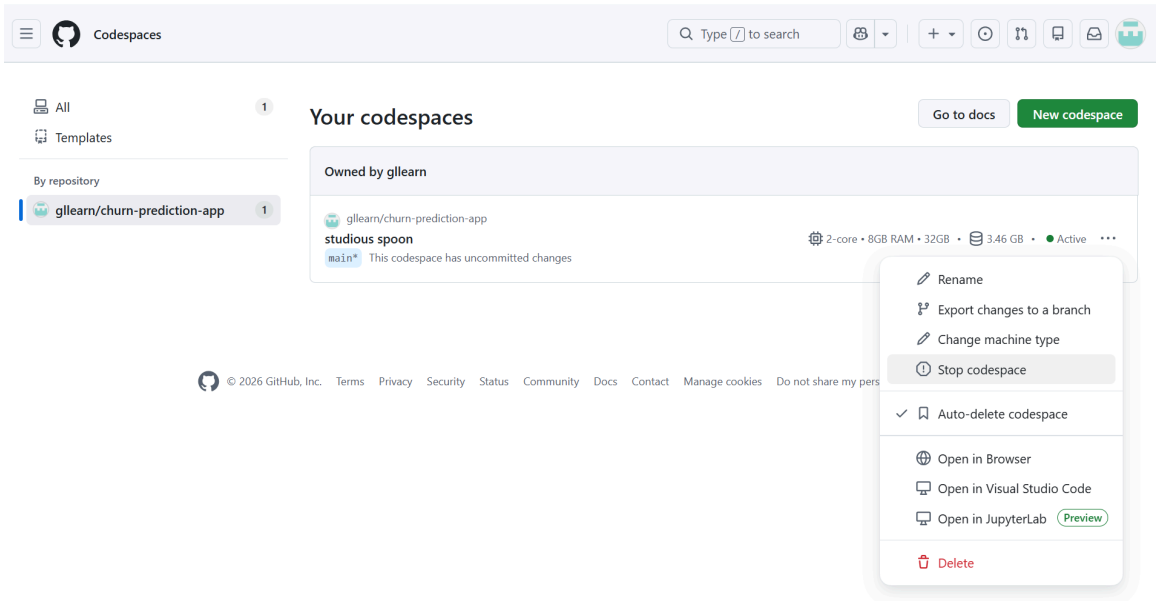
Click the three dots on the right side of your codespace entry to see management options. You can rename it, export changes to a branch, change the machine type, or stop the codespace when you are done working.

Please make sure to stop the codespace when you are not using it. You can also enable auto-delete so it gets cleaned up automatically after a period of inactivity.

Note: As of July 14, 2026, GitHub Codespaces provides 60 hours of usage per month for a 2-core CPU configuration. Hence, it is highly advisable to stop the Codespace whenever it is not in use to maximize the available usage hours and get the maximum benefit from the free-tier allowance.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

To permanently remove it, scroll to the bottom of the menu and click "Delete."



Wrapping Up

That covers the full process from start to finish. To recap: you create a codespace from your repository, install the dependencies, run the Streamlit command, make the port public, and your app is live. No local setup, no server configuration, no deployment pipelines to worry about.

Just remember to stop or delete your codespace when you are done so you do not burn through your free usage hours unnecessarily.

This file is meant for personal use by alienrivero@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.